



AgriPrice Intelligence

Smart Market Analytics for Farmers

Project Presentation Showcase 2026

Prepared by: Neti Geethika

GitHub: github.com/Neti-Geethika/AgriPrice-Intelligence

1. Problem Statement

Farmers across India frequently sell their produce at a loss — not because of poor crop quality, but because they lack visibility into price trends, seasonal patterns, and better-paying nearby markets. Prices for the same crop can vary significantly between mandis just a few kilometers apart, and can swing sharply due to seasonality, oversupply, or demand shocks — information most farmers have no easy way to access.

AgriPrice Intelligence addresses this by turning real government mandi price data into two things: a Power BI dashboard for deep market analysis, and a simple farmer-facing app that answers one direct question — where should I sell, and what will I get for it?

Why This Project

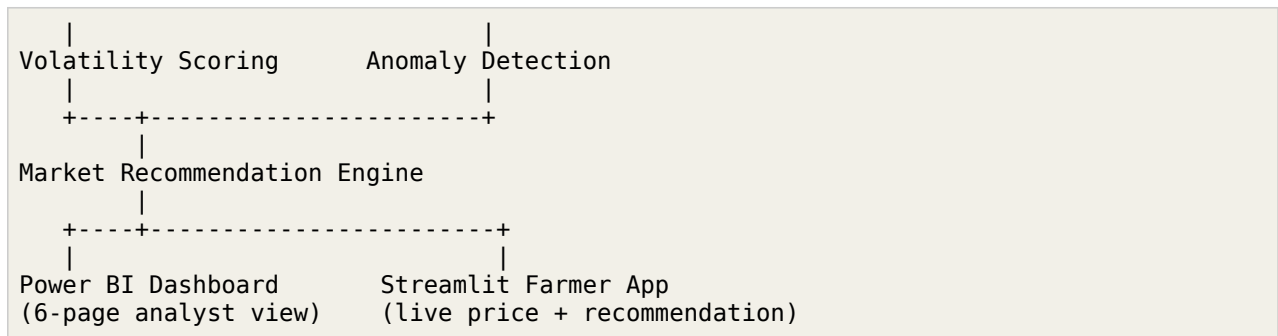
- **Real-world relevance:** uses actual government open data (data.gov.in), not a synthetic or Kaggle dataset — the problem and the data are both genuine.
- **Two audiences, one model:** an analyst dashboard (Power BI) and a farmer-facing app (Streamlit) are both built on the same trained ML pipeline, showing the same intelligence delivered two different ways for two different users.
- **End-to-end scope:** covers data collection via a live API, cleaning, EDA, feature engineering, ML modeling, explainability, and two deployed interfaces — a genuinely complete pipeline, not just a notebook.

2. Objectives

- Predict future crop prices using historical market data
- Explain what drives each prediction (not just output a number)
- Classify crops by price volatility / risk
- Detect unusual historical price events (crashes/spikes)
- Recommend the best nearby market to sell in, based on predicted price
- Fetch and display live/current government-reported prices
- Present the same intelligence to two audiences: analysts (Power BI) and farmers (Streamlit app)

3. System Architecture

```
Agmarknet API (data.gov.in)
|
Data Cleaning & Validation (Python, Pandas)
|
EDA – trends, seasonality, market spread
|
Feature Engineering – lag price, rolling avg, season, day-of-week
|
ML Price Prediction (Linear Regression vs Random Forest vs XGBoost)
|
SHAP Explainability
|
+-----+
```



4. Data Collection

Source

Ministry of Agriculture & Farmers Welfare — Directorate of Marketing and Inspection (DMI), published via data.gov.in under the dataset "Variety-wise Daily Market Prices Data of Commodity" (Agmarknet).

How It Was Collected

- **Step 1:** Registered for a free data.gov.in API key (universal key, works across all OGD Platform datasets).
- **Step 2:** Identified the resource endpoint and confirmed available fields: Arrival_Date, State, District, Market, Commodity, Variety, Grade, Min_Price, Max_Price, Modal_Price.
- **Step 3:** Wrote a Python script (requests + pandas) to pull data in paginated batches (limit/offset), looping across multiple districts and states.
- **Step 4:** Debugged real-world API issues along the way — including a 'Key not authorised' error (resolved by waiting for key activation) and repeated connection timeouts (resolved by adding a browser-style User-Agent header, since the API was silently rejecting non-browser-like requests).
- **Step 5:** Pulled data for 3 target states (Andhra Pradesh, Telangana, Karnataka) across 15 districts; found Andhra Pradesh's available records did not fall within the last 2 years, so it was excluded to keep data recency consistent.
- **Final coverage:** Telangana & Karnataka, 9 districts, ~19,373 cleaned records after filtering to 10 target crops, spanning 2024–2026.

Target Crops Selected

Onion, Tomato, Potato, Rice, Maize, Cabbage, Carrot, Brinjal, Banana, Mango — chosen for strong data coverage and a mix of staples, vegetables, and fruit for a relatable, diverse demo.

5. Data Cleaning

- Parsed and standardized Arrival_Date into proper datetime format
- Converted Min_Price, Max_Price, Modal_Price to numeric; dropped rows with invalid/missing prices
- Removed rows with zero/negative prices (data entry errors)
- Removed rows where Max_Price < Min_Price (logical inconsistency)

- Standardized text fields (Commodity, District, Market — trimmed whitespace)
- Removed duplicate rows
- Result: only ~15 of 19,388 rows (0.08%) were dropped — confirming government data was already high quality

6. Exploratory Data Analysis

EDA answered these core questions before any modeling was attempted:

- How does price trend over time, per crop?
- What seasonal patterns exist (month-wise average price)?
- How much does price vary across markets for the same crop?
- Which crops are most vs. least volatile?

This step surfaced an important data-quality insight: the raw dataset (all 78 commodities, including livestock like 'He Buffalo' and 'She Goat') had unreliable volatility numbers for low-frequency items. This directly justified narrowing the project to 10 well-represented target crops rather than analyzing everything.

7. Feature Engineering

Features built for the ML model, sorted by Commodity + Market + Date to preserve chronological order:

```
month, day_of_week, season
lag_price_1      (previous recorded price, same crop+market)
lag_price_7      (price 7 records earlier)
rolling_avg_7    (7-record rolling average price)
price_change_pct (% change from previous record)
```

Rows with no lag_price_1 (first record per crop+market group) were dropped, since lag features are undefined there. Final feature-ready dataset: 4,900 rows.

8. Machine Learning: Price Prediction

Methodology

- **Time-based split, not random:** trained on earlier dates, tested on later dates (80/20), to avoid leaking future information into training — critical for honest evaluation on time-series data.
- **Naive baseline first:** predicting 'yesterday's price' as the forecast, to establish the real bar any model needs to beat.
- **Three models compared:** Linear Regression, Random Forest, XGBoost.

Results

Model	MAE	RMSE	R ²
Naive Baseline (last known price)	617.30	1364.07	—

Linear Regression (CHOSEN)	464.86	867.04	0.704
XGBoost (tuned, shallow)	477.74	904.30	0.678
Random Forest (tuned, shallow)	492.11	914.74	0.670
Random Forest (default)	557.57	1057.30	0.560
XGBoost (default)	581.48	1083.48	0.538

Why Linear Regression Was Chosen

Tree-based models (Random Forest, XGBoost) underperformed the simpler Linear Regression at default settings — unusual, since trees typically outperform linear models. Investigation showed this was overfitting on the dataset's size (~3,900 training rows across many categorical splits). This was tested directly: shallower/tuned versions of both tree models were trained, and their error dropped substantially (XGBoost MAE improved from 581 to 478), closing most — but not all — of the gap with Linear Regression. This confirmed the diagnosis and validated that Linear Regression's win was genuine generalization, not a fluke.

All models beat the naive baseline (MAE 617.30), with the chosen model improving on it by roughly 25%.

9. Explainability (SHAP)

SHAP (SHapley Additive exPlanations) was applied to the trained Linear Regression model to determine what actually drives each price prediction.

Feature	Mean SHAP value
rolling_avg_7	1029.50 <- dominant driver
lag_price_1	135.30
Month	15.50
DayOfWeek	11.01
Commodity	10.90
Season	9.77
District	4.59
Market	2.32

Interpretation: predictions are driven overwhelmingly by recent price momentum (the 7-day rolling average), roughly 7x more influential than every other feature combined. This means the model is strong at short-term trend continuation, but — by design — cannot anticipate sudden shocks (unexpected oversupply, policy changes, extreme weather), since none of those are in the feature set. This limitation is stated openly rather than hidden.

10. Volatility & Risk Scoring

Volatility was calculated per crop using the coefficient of variation (CV = standard deviation ÷ mean price), then classified into risk bands:

- Low risk: CV < 0.30 (e.g., Brinjal)

- Medium risk: CV 0.30–0.45 (e.g., Potato, Maize, Mango, Cabbage, Banana, Carrot)
- High risk: CV > 0.45 (e.g., Rice, Onion, Tomato)

This risk label is surfaced directly in the farmer app so a farmer immediately understands how much a crop's price typically swings.

11. Anomaly Detection

Isolation Forest was applied per crop (not globally, since a 'normal' price for Rice is very different from Onion) to flag unusual historical price events.

A concrete example surfaced by this step: Tomato prices in Miryalguda (Rythu Bazar) crashed to around ₹800/quintal in March–April 2022, then spiked to ₹5,800–6,300/quintal by June 2022 — a genuine, visually compelling oversupply-then-shortage story used directly in the live demo.

12. Market Recommendation Engine

For a chosen crop, the system takes each market's most recent known data, runs it through the trained model, and ranks markets by predicted price (not just current price) — so the recommendation reflects where prices are trending, not just a snapshot.

Example output for Onion: Belgaum APMC predicted ₹2,744/quintal (vs. last known ₹2,700), ranked above Mysuru APMC and Bengaluru APMC.

13. Live Price Fetch

A dedicated function queries the Agmarknet API live for a chosen state/district/commodity and returns the most recent record. A freshness check was added after testing revealed the API sometimes returns records that are years old for certain crop/district combinations (e.g., a Tomato/Warangal query returned a 2022 record, over 1,500 days old).

- **Freshness rule:** records within 90 days are treated as 'recent' and shown as the current price; older records trigger a visible warning in the app, and the system falls back to the model's prediction instead of silently presenting stale data as current.

14. Streamlit Farmer-Facing App

A single-screen, plain-language interface designed for a non-technical user standing in a field with a phone — not a dashboard, an answer.

Flow

- Farmer selects their crop and district from two dropdowns
- Clicks 'Get Recommendation'
- App shows: latest reported price (with freshness warning if stale), best predicted price, a color-coded risk badge (green/amber/clay-red), a ranked list of top 5 nearby markets, and a recent price trend chart

Design

Custom-styled with a 'harvest field' visual identity — deep forest green header, warm amber accents, editorial serif headings (Fraunces) paired with a clean sans body (Inter) — built with Streamlit + custom CSS + Plotly for the interactive chart.

15. Power BI Dashboard (Analyst View)

A 6-page dashboard built on an exported dataset containing actual prices, model predictions, prediction error, anomaly flags, and volatility/risk labels — enabling both descriptive and predictive analysis in one file.

- **Page 1 — Overview:** total records, average price, anomaly count, risk breakdown table, slicers for Commodity/District/State.
- **Page 2 — Price Trends:** monthly average price line chart per crop, and a seasonality chart (month-wise average).
- **Page 3 — Price Predictor:** predicted vs. actual price line chart, drill-through by Commodity/Market, model accuracy % card.
- **Page 4 — Volatility & Risk:** bar chart of volatility score by crop, colored by risk level; matrix view of crop vs. risk level.
- **Page 5 — Anomaly Timeline:** price-over-time chart highlighting flagged anomaly points, defaulted to Tomato to show the 2022 crash-then-spike event.
- **Page 6 — Regional Map:** geographic view of average price by district for a selected crop.

16. Tools & Tech Stack

Layer	Tool	Purpose
Data source	data.gov.in (Agmarknet API)	Real government mandi price records
Data validation	Excel	Quick pivot checks on raw pulls
Data handling	Python, Pandas, NumPy	Cleaning, merging, feature engineering
ML modeling	Scikit-learn, XGBoost	Price prediction, anomaly detection
Explainability	SHAP	Feature importance behind each prediction
Visualization (app)	Plotly	Interactive price trend chart
Farmer-facing app	Streamlit	Live, plain-language recommendation tool
Analyst dashboard	Power BI	6-page deep-dive dashboard with DAX & maps
Version control	Git / GitHub	Commit history, code hosting

17. Honest Limitations

- **Uneven live-data freshness:** government mandi reporting isn't uniform — some crop/district combinations have recent data, others don't. The app surfaces this explicitly rather than hiding it.
- **Trend-following, not shock-predicting:** the model is strong at short-term continuation but cannot anticipate weather events, policy changes, or sudden oversupply — this is inherent to the feature set, not a bug.
- **Andhra Pradesh excluded:** its available records didn't fall within the last 2 years; rather than mixing inconsistent recency across states, it was dropped to keep the dataset consistent.
- **Uneven district/crop coverage:** some crop-district combinations have very few records (e.g., a handful of Banana records outside Bengaluru), so market comparisons for those are naturally thinner — a realistic reflection of where each crop is actually traded, not a flaw to hide.

18. Anticipated Q&A

Below are the questions most likely to come up, with strong, honest answers grounded in the actual build process.

Q: Why did you choose Linear Regression over Random Forest or XGBoost?

A: Tree-based models underperformed at default settings — unusual, since trees typically beat linear models. I tested shallower, tuned versions of both to check for overfitting, and their error improved substantially, confirming the diagnosis. Linear Regression still generalized best on the held-out time-based test set, so I picked the model that actually performs best, not the most complex one.

Q: How did you validate your model honestly?

A: I split the data by time — training on earlier dates and testing on later ones — rather than a random split, since random shuffling would leak future information into training on time-series data. I also benchmarked every model against a naive 'yesterday's price' baseline to prove the model adds real value, not just noise-fitting.

Q: What does your SHAP analysis tell you?

A: Predictions are driven overwhelmingly by the 7-day rolling average price — about 7 times more influential than every other feature combined. This means the model is excellent at trend continuation but, by design, can't foresee sudden shocks like unexpected oversupply or policy changes, since those aren't captured in the feature set.

Q: Why did you exclude Andhra Pradesh?

A: Its available data didn't fall within the last 2 years. Rather than mixing states with inconsistent data recency — which would make live/current recommendations unreliable for one state but not others — I kept the dataset consistent to Telangana and Karnataka.

Q: How 'live' is your live price data, really?

A: I built a freshness check into the live fetch function. If a government-reported record is more than 90 days old, the app flags it explicitly and falls back to the model's prediction instead of presenting stale data as current — this came directly from testing, where I found some crop/district queries returned records years old.

Q: How does the market recommendation actually work?

A: For a selected crop, I take each market's most recent known data, run it through the trained price prediction model, and rank markets by predicted price rather than just the last known price — so the recommendation reflects where prices are trending, not a stale snapshot.

Q: What's the difference between the Power BI dashboard and the Streamlit app?

A: Both run on the same trained model and dataset, but serve different users. Power BI is the analyst view — deep trend analysis, drill-throughs, geographic comparison. The Streamlit app is built for a farmer: two dropdowns, one button, and a plain-language answer — no charts to interpret unless they want to see the trend line.

Q: How did you handle data quality issues?

A: Cleaning removed zero/negative prices, rows where max price was less than min price, and duplicates — only about 0.08% of rows were dropped, showing the government data was largely reliable. I also found the raw 78-commodity dataset included non-crop items like livestock, which skewed volatility statistics — that directly justified narrowing scope to 10 well-represented target crops.

Q: What was the hardest technical problem you solved?

A: Getting the government API to respond reliably in Python. It worked fine in a browser but timed out or reset the connection every time from Python — I isolated it to a missing browser-style User-Agent header, which the API was silently using to filter out non-browser requests. Adding that header fixed it.

Q: What would you improve with more time?

A: I'd add more states with better recent coverage, incorporate weather data as a feature to catch some of the 'shock' events the current model misses, and build a proper scheduled data refresh instead of an on-demand live fetch.

Q: Is this actually useful to a real farmer, or just a technical exercise?

A: That was the explicit design goal — the Streamlit app deliberately avoids charts and jargon in its main flow. A farmer picks their crop and district and gets a direct answer: today's price, the best nearby market, and whether prices are trending up or down. The deeper analysis lives in Power BI for anyone who wants to dig further.